

histograms

ConstantFieldError

Bases: Exception

Raised when numerical field is constant.

NumericalHistogramConfig

```
NumericalHistogramConfig(minv, maxv, max_bins=1000)
```

Bases: object

Configuration object for numerical histograms.

It requires three configurable values: minv/maxv the minimum and maximum histogram values, and max_bins the number of bins or resolution of the histogram.

The minv and maxv configurables are private accessible via get_minmax() and set_minmax(), while nbins is a property.

Parameters:

Name	Type	Description	Default
minv		minimum value of the histogram. Smaller values will be counted in the underflow bin.	required
maxv		maximum value of the histogram. The last bin is closed on the right side, i.e. entries with value == maxv will be counted in the last bin. Larger values will be counted in the overflow bin.	required
max_bins		resolution of the histogram, i.e. number of bins within minv and maxv. There will be two extra bins to account for underflow and overflow.	1000

max_bins property

```
max_bins
```

get_minmax

```
get_minmax()
```

set_minmax

```
set_minmax(minv, maxv)
```

Setter function for minv and maxv. It raises exceptions if they are not finite numbers or if minv > maxv.

TimeHistogramConfig

```
TimeHistogramConfig(time_resolution='h')
```

Bases: object

Configuration object for datetime histograms. It requires one configurable value: the `time_resolution` of the plot.

Parameters:

Name	Type	Description	Default
<code>time_resolution</code> ↗		time resolution of the histogram.	<code>'h'</code>

`time_resolution` [property](#) [writable](#) [↗](#)

`time_resolution`

`build_numerical_bin_expr` [↗](#)

```
build_numerical_bin_expr(field: StructField, stats: dict, max_bins: int) -> Tuple[str, Column, Callable] | None
```

Builds a bin expression for a numerical field.

Returns:

Type	Description
tuple of (str, Column, Callable)	(<code>field_name</code> , <code>bin_expr</code> , <code>decode_fn</code>) where <code>bin_expr</code> maps each value to a string-encoded bin index and <code>decode_fn</code> converts the string back to the (<code>bin_range</code> , <code>breakdown_values</code>) key used in the histogram dict. If the field is constant, returns <code>None</code> .

`build_string_bin_expr` [↗](#)

```
build_string_bin_expr(field: StructField, stats: dict, cardinality_threshold: int | None, dist_sampling: float | None) -> Tuple[str, Column, Callable]
```

Builds a bin expression for a string field.

Returns:

Type	Description
tuple of (str, Column, Callable)	(<code>field_name</code> , <code>bin_expr</code> , <code>decode_fn</code>).

`build_timestamp_bin_expr` [↗](#)

```
build_timestamp_bin_expr(field: StructField, time_config: TimeHistogramConfig) -> Tuple[str, Column, Callable]
```

Builds a bin expression for a timestamp field.

Returns:

Type	Description
tuple of (str, Column, Callable)	(<code>field_name</code> , <code>bin_expr</code> , <code>decode_fn</code>).

compute_all_histograms [↗](#)

```
compute_all_histograms(data: _TDataFrame, field_bin_exprs: list[Tuple[str, Column, Callable]], breakdown_cols: list[Column], breakdowns: list[str]) -> dict[str, dict]
```

Compute histograms for all fields in a single pass:

1. Builds an array-of-structs with one element per field, each containing `(field_name_literal, bin_expression)`.
2. Explodes the array so every input row produces N rows (one per field).
3. Runs a single `groupBy("__field", "__bin", *breakdowns).count()`.
4. Partitions the collected rows back into per-field histogram dicts using the field-specific `decode_fn`.

Parameters:

Name	Type	Description	Default
<code>data</code> ↗	<code>DataFrame</code>	Source <code>DataFrame</code> .	<i>required</i>
<code>field_bin_exprs</code> ↗	list of (str, Column, callable)	One tuple per field, as returned by <code>build_*_bin_expr</code> . Each tuple is <code>(field_name, bin_column_expr, decode_fn)</code> .	<i>required</i>
<code>breakdown_cols</code> ↗	list of Column	Pre-built breakdown column expressions for the <code>groupBy</code> .	<i>required</i>
<code>breakdowns</code> ↗	list of str	Field names corresponding to <code>breakdown_cols</code> .	<i>required</i>

Returns:

Type	Description
<code>dict[str, dict]</code>	Mapping from field name to histogram dict. Each histogram dict maps <code>(bucket_key, breakdown_values) -> count</code> .